

Channel and Flow

Реактивное программирование

Организационные вопросы(снова)

Порядок проверки

- На каждую работу предусмотрено до 2 итераций ревью.
- Если после второй итерации работа набирает 6+ баллов и является минимально приемлемой, я принимаю её автоматически.
- Если после двух итераций работа всё ещё слабая, решение о дальнейших доработках остаётся на моё усмотрение.
- Если работа уже принята, отдельная повторная проверка новых исправлений не предусмотрена и возможна только по личной договорённости.

Порядок проверки Доработки

- после принятия работы у вас есть 30 дней на доработку;
- если доработка не прислана в этот срок, работа фиксируется в текущем состоянии;

Порядок проверки

- если вы претендуете на 9-10 баллов, недостаточно просто закрыть замечания из ревью. Высокий балл предполагает, что работа в целом сделана сильно и аккуратно, в том числе за пределами прямо указанных исправлений.
- Поэтому, если вы рассчитываете на высокий балл, важно самостоятельно проверить работу целиком: аккуратность реализации, качество кода, обработку сценариев, соответствие заданию и общее впечатление от результата.

Ревью - это не бесконечный цикл правок.

У вас есть возможность исправить основные замечания, но ответственность за качество работы лежит на вас, а не только на списке моих комментариев.

**После launch/async возникает
НОВЫЙ вопрос**

**Не “как запустить”, а “как общаться и
как наблюдать данные”**

Проблема

Корутины сами по себе не отвечают на 3 проблемы

Shared mutable state

Две корутины меняют один объект.
Появляются гонки и синхронизация.

Сигналы и события

Надо передать “сделай это”,
“вот новый элемент”, “мы закончили”.

Поток данных во времени

Нужна не одна величина, а
последовательность значений:
новости, поиск, база, UI state.

Reactive Programming

Императивный стиль

```
var a = 2  
var b = 3  
var c = a + b
```

```
println(c) // 5
```

```
a = 10  
println(c) // всё ещё 5
```


Пример в таблицах

Что такое реактивное программирование?

- Реактивное программирование - это стиль разработки
- Программа работает с потоками событий и данных
- Код реагирует на новые значения, ошибки и завершение

Что такое реактивное программирование?

- Императивно: “дай мне значение”
- Реактивно: “сообщай мне об изменениях”
- Один ответ vs поток обновлений

Реактивное программирование - это когда мы работаем не просто со значениями, а с изменениями значений во времени.

А что значит во времени?

Простая интуиция

```
fun getTemperature(): Int = 22
```

Но мы хотим 22 -> 23 -> 22 -> 21 -> 20

Главный мысль

- функция `getTemperature()` отвечает на вопрос “какая температура сейчас?”
- Но реактивный поток отвечает на другой вопрос:
“сообщай мне каждый раз, когда температура меняется”.

Ты сам решаешь, когда спрашивать

```
while (true) {  
    val t = getTemperature()  
    println(t)  
    Thread.sleep(1000)  
}
```


Ты сам решаешь, когда спрашивать

```
while (true) {  
    val t = getTemperature()  
    println(t)  
    Thread.sleep(1000)  
}
```

Но сразу возникает вопрос:

а почему раз в секунду?

- а если температура изменилась через 100 мс?
- а если она не меняется 10 минут?
- а если надо раз в 5 секунд?
- а если частота должна меняться?

То есть при подходе с функцией ты вынужден сам придумывать **политику опроса**.

Ты сам решаешь, когда спрашивать

Допустим, температура менялась так:

22 -> 23 -> 24 -> 23

Если ты спрашиваешь раз в секунду, то можешь увидеть только:

22 -> 23

Функция отвечает на вопрос: “какое значение сейчас?”

Реактивный поток отвечает на вопрос: “как это значение ведёт себя со временем?”

The Reactive Manifesto

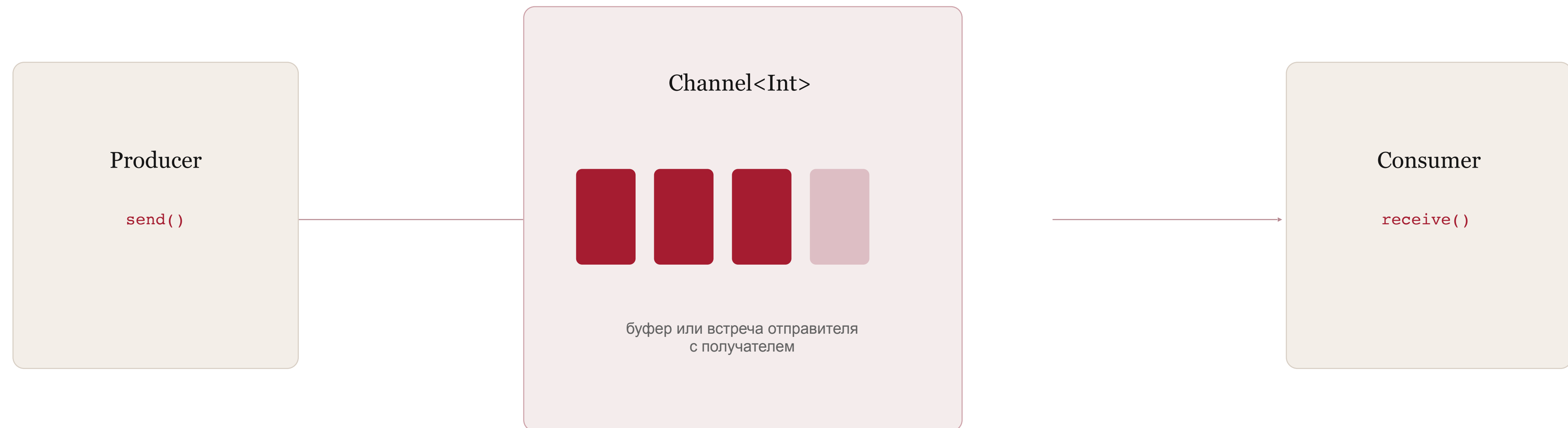
- ОТЗЫВЧИВЫМИ,
- УСТОЙЧИВЫМИ К СБОЯМ,
- ЭЛАСТИЧНЫМИ,
- и message-driven.

<https://www.reactivemanifesto.org/>

Channel

Channel = mailbox между корутинами

Похоже на BlockingQueue, но send/receive - suspending, а не blocking



- Хорош для координации: команды, задания, элементы очереди
- Не хранит "состояние экрана" как главную модель мира
- Может быть закрыт: получатель поймёт, что поток элементов завершён

Главная свойство Channel - capacity

Поведение канала меняется от размера буфера и стратегии переполнения



Практический ориентир: channel выбирают не только по типу данных, но и по тому, как должны вести себя producer и consumer при разной скорости.

Минимальный Channel: producer -> consumer

где “очередь задач” понятнее, чем shared mutable state

```
1 val ch = Channel<Int>(capacity =  
Channel.BUFFERED)  
2  
3 launch {  
4     repeat(5) {  
5         ch.send(it)  
6     }  
7     ch.close()  
8 }  
9  
10 launch {  
11     for (x in ch) {  
12         println(x)  
13     }  
14 }
```

- send() приостанавливает producer, если канал не готов принять элемент
- for (x in ch) удобно читает элементы до close()
- close() сигнализирует: больше значений не будет
- Такой код естественно расширяется в fan-in / fan-out схемы

Channel - не новое “универсальное состояние приложения”

Сильный инструмент координации, слабый инструмент долговременного состояния

Где Channel уместен

- Очередь заданий между coroutines
- Единичные события и команды
- Pipeline: producer → stage → consumer
- Backpressure через capacity и suspension

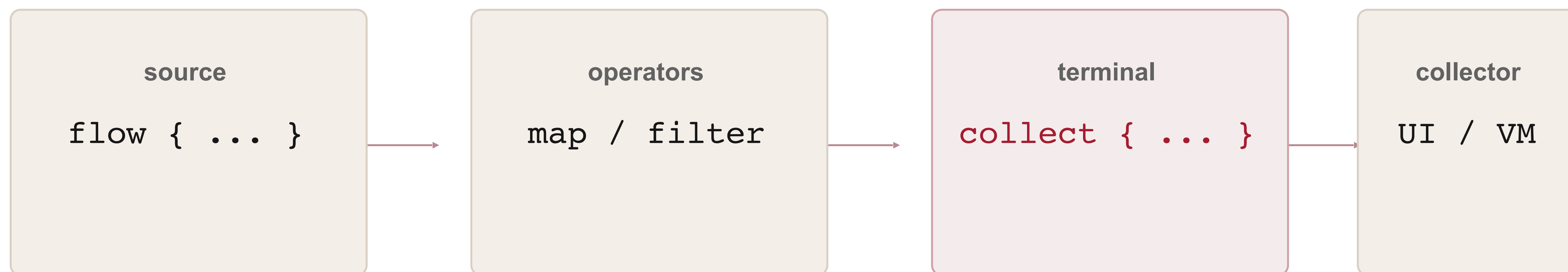
Где он создаёт лишнюю боль

- Хранение текущего UI state
- Повторные подписки “дай мне последнее значение”
- Сложная replay-логика поверх событий
- Замена Flow без реальной причины

Flow

Flow = асинхронный поток значений

Обычно cold: код внутри source запускается только при collect()



- Flow умеет отдавать несколько значений последовательно
- Cold-природа означает: каждый collect заново запускает upstream
- Операторы создают конвейер трансформаций, а не немедленный результат

Cold or Hot

Flows are cold

```
fun simple(): Flow<Int> = flow {  
    println("Flow started")  
    for (i in 1..3) {  
        delay(100)  
        emit(i)  
    }  
}  
  
fun main() = runBlocking<Unit> {  
    println("Calling simple function...")  
    val flow = simple()  
    println("Calling collect...")  
    flow.collect { value -> println(value) }  
    println("Calling collect again...")  
    flow.collect { value -> println(value) }  
}
```


Flows are cold

```
fun simple(): Flow<Int> = flow {  
    println("Flow started")  
    for (i in 1..3) {  
        delay(100)  
        emit(i)  
    }  
}  
  
fun main() = runBlocking<Unit> {  
    println("Calling simple function...")  
    val flow = simple()  
    println("Calling collect...")  
    flow.collect { value -> println(value) }  
    println("Calling collect again...")  
    flow.collect { value -> println(value) }  
}
```

Calling simple function...

Calling collect...

Flow started

1

2

3

Calling collect again...

Flow started

1

2

3

Минимальный Flow: builder -> operators -> collect

Один и тот же код читается почти как “обычный” pipeline данных

```
1 val numbers = flow {  
2     emit(1)  
3     delay(100)  
4     emit(2)  
5     emit(3)  
6 }  
7  
8 numbers  
9     .map { it * 10 }  
10    .filter { it >= 20 }  
11    .collect { println(it) }
```

Что здесь важно

- emit() отправляет очередное значение в поток
- map / filter описывают преобразование декларативно
- Ничего не происходит, пока не появился terminal operator - collect()
- Если собрать numbers дважды, upstream выполнится заново

How stream: горячий поток

- Источник живёт независимо от подписчиков
- Подписчик подключается к уже идущему источнику
- Несколько подписчиков могут делить один upstream

SharedFlow и StateFlow

- SharedFlow - hot broadcast stream
- StateFlow - hot stream состояния
- Они живут независимо от collector'ов

SharedFlow

- SharedFlow - есть один общий источник, и все подписчики получают одни и те же события в broadcast-стиле. broadcast-ит значения всем своим collectors, и его активный экземпляр существует независимо от наличия подписчиков.
- есть настраиваемые параметры:
 - replay - сколько последних значений сразу отдать новому подписчику;
 - extraBufferCapacity - дополнительный буфер;
 - onBufferOverflow - что делать при переполнении: suspend/drop и т.д.

StateFlow

- StateFlow - это специализированный SharedFlow для состояния. Документация говорит: это SharedFlow, который хранит одно текущее значение state, доступное через .value, и он тоже hot - существует независимо от collectors.
- Ключевые свойства StateFlow:
 - у него всегда есть initial value;
 - новый подписчик сразу получает последнее текущее значение;
 - он хранит последнее состояние, а не историю событий;
 - обновления conflated: медленный collector может пропустить промежуточные быстрые апдейты, но всегда увидит самое новое состояние.

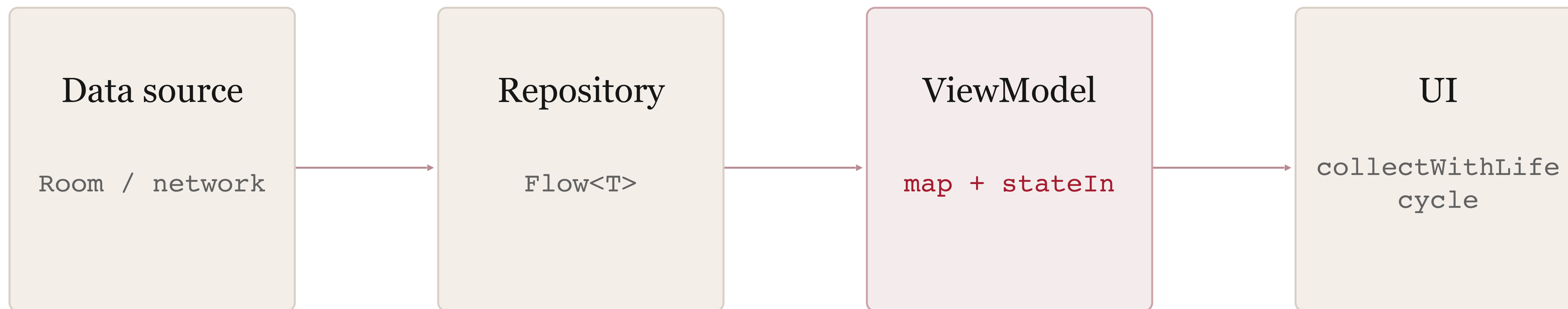
State vs Event

- `StateFlow` = состояние
- `SharedFlow` = события
- Это не одно и то же

Flow на Android

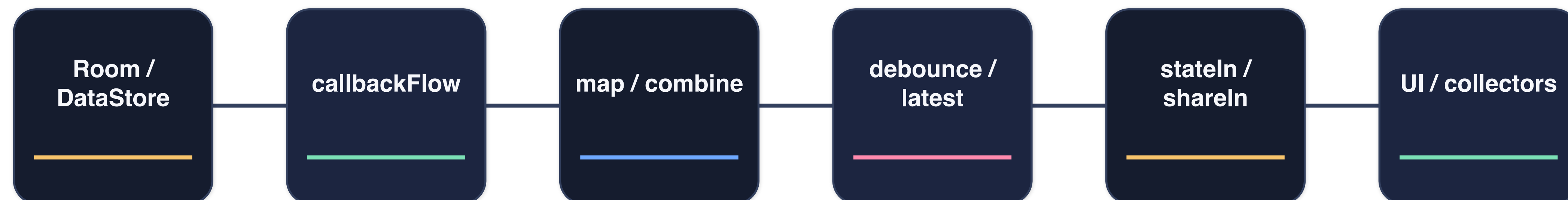
Flow на Android: естественная модель наблюдаемых данных

Repository публикует поток значений, ViewModel его преобразует, UI подписывается по lifecycle



- Flow особенно хорош там, где данные меняются со временем
- ViewModel обычно превращает cold Flow из data layer в удобный hot state для UI
- UI не “спрашивает заново”, а реагирует на новые значения

Где Flow в Android действительно уместен



FLOW НУЖЕН

- поиск: запрос -> debounce -> distinctUntilChanged -> flatMapLatest
- комбинация нескольких источников: Room + фильтры + сеть
- шаринг дорогого upstream: один callback / один сокет на всех

ПОТОМУ ЧТО

- периодический refresh / tick на несколько модулей
- экранный state из stream-источников через stateIn
- реактивная отмена предыдущей работы при новом значении

Главная мысль

Не любой state в приложении обязан становиться Flow

Если на экране нет реального реактивного pipeline, то StateFlow часто - лишний уровень сложности.

Для простого UI element state в Compose часто безопаснее и дешевле использовать mutableStateOf / rememberSaveable.

mutableStateOf

Локальный или UI-only state.
Читаешь value — и всё.

минимум
бойлерплейта

StateFlow

Нужен current state + коллекторы + pipeline.

для потоков
состояния

SharedFlow

Hot broadcast без обязательного initial value.

для событий/
шаринга

Быстрый выбор инструмента

	<code>mutableStateOf</code>	<code>StateFlow</code>	<code>SharedFlow</code>
Что моделирует	UI element state	Текущее состояние	Broadcast / события / shared stream
Как UI читает	Прямо value / by	<code>collectAsStateWithLifecycle</code>	<code>LaunchedEffect { collect }</code>
Initial value	Обычно есть	Обязателен	Не обязателен
Новый подписчик	Читает текущее value	Получает последнее value	Получает replay, если настроен
Collector overhead	Нет отдельного collect	Есть	Есть

StateFlow: операторы

Обычный пример: reactive search pipeline

ViewModel

```
1 private val query =  
MutableStateFlow("")  
2  
3 val uiState = query  
4     .debounce(300)  
5     .filter { it.length >= 2 }  
6     .distinctUntilChanged()  
7     .flatMapLatest(repository::search)  
8     .map(::toUiState)  
9     .stateIn(  
10         viewModelScope,  
11  
SharingStarted.WhileSubscribed(5_000),  
12         SearchUiState.Empty  
13     )
```

debounce

distinctUntil
Changed

flatMapLatest

map

stateIn

- StateFlow всегда имеет initial value и сразу отдаёт текущее состояние новому collector.
- Нормальный use case: экранный uiState, собранный из Flow-источников ниже по слоям.
- Через value состояние можно прочитать синхронно - это удобно для тестов и императивных проверок.
- Но если pipeline нет, превращать обычный state в MutableStateFlow - спорное усложнение.

SharedFlow

отдельный инструмент для hot broadcast

вариант 1 - внутренние события / тик

```
1 private val _ticks =  
MutableSharedFlow<Unit>(  
2     replay = 0,  
3     extraBufferCapacity = 1,  
4     onBufferOverflow =  
BufferOverflow.DROP_OLDEST  
5 )  
6 val ticks = _ticks.asSharedFlow()
```

вариант 2 — шаринг холодного upstream

```
1 val updates = repository.upstream  
2     .shareIn(  
3         scope = appScope,  
4         started =  
SharingStarted.WhileSubscribed(),  
5         replay = 1  
6     )
```

- Есть несколько consumers и всем нужен один и тот же hot stream.
- Нужен broadcast без обязательного initial value.
- Нужен replay / buffer / overflow policy под задачу.
- shareIn полезен, когда upstream дорогой и не должен стартовать заново на каждый collect.

Осторожно: для one-shot UI событий SharedFlow тоже может быть ловушкой - late collector, replay, duplicate delivery. В Compose часто чище выразить эффект через state + LaunchedEffect.

StateFlow как обычная переменная экрана

Когда реальной реактивности нет

шаблон, который часто пишут «по привычке»

```
1 private val _searchState =  
2 MutableStateFlow<SearchState>(SearchState.E  
  mpty)  
3 val searchState: StateFlow<SearchState> =  
4     _searchState.asStateFlow()  
5  
6 fun onTextChanged(text: String) {  
7     _searchState.value =  
8     SearchState.Input(text)  
9 }
```

если это просто UI element state

```
1 var query by mutableStateOf("")  
2 private set  
3  
4 fun onTextChanged(text: String) {  
5     query = text  
6 }  
7  
8 // UI просто читает query
```

ПОЧЕМУ ПЛОХО

- Появляется лишний collector и lifecycle-связка там, где можно было просто прочитать значение.
- Больше бойлерплейта: backing property, asStateFlow(), collectAsStateWithLifecycle().
- Больше мест для ошибок: не тот scope, не тот lifecycle, лишние подписки, event replay.
- Тестировать и дебажить сложнее: корутины, virtual time, отдельные правила для stateIn/shareIn.
- Команда без опыта в реактивном стиле начинает копировать паттерн механически.

Исключение: если этот state реально комбинируется с другими Flow или нужен outside Compose - тогда StateFlow уже оправдан.

Самые частые ловушки с collect

1. collect прямо из launch в UI

Проблема: flow продолжает обрабатываться, даже когда view не видна.

Фикс: `repeatOnLifecycle()` или `collectAsStateWithLifecycle()`.

2. collect в теле Composable

Проблема: recomposition может создать повторные collectors.

Фикс: `LaunchedEffect(key) { flow.collect { ... } }`.

3. SharedFlow как one-shot event

Проблема: late collector, неверный replay, потеря или дублирование события.

Фикс: осознанно настраивать replay/buffer или моделировать эффект через state.

4. collect vs collectLatest

Проблема: `collectLatest` отменяет предыдущую обработку при новом значении.

Фикс: использовать только если отмена действительно ожидаема.

Ловушки stateIn / shareIn / SharedFlow

- stateIn + WhileSubscribed в тестах не стартует без хотя бы одного collector.
- StateFlow подавляет повторную эмиссию equal-значений: это удобно, но может ломать ожидания.
- SharedFlow без replay/buffer легко теряет события для позднего subscriber.
- shareIn/stateIn требуют scope, который живёт дольше consumer; иначе upstream будет стартовать/умирать не там, где ты ждёшь.
- collectLatest / flatMapLatest / mapLatest отменяют предыдущую работу - иногда это плюс, иногда скрытый баг.
- Чем больше collectors, тем дороже апдейты и тем сложнее reasoning о жизненном цикле.

StateFlow и SharedFlow

Когда нужен не просто Flow, а разделяемый источник значений

StateFlow

- Всегда имеет текущее value
- Идеален для UI state
- Новый collector сразу получает последнее состояние

```
stateIn(...  
)
```

SharedFlow

- Горячий поток для нескольких подписчиков
- Подходит для широковещательных событий
- Можно настроить replay и extra buffer

```
shareIn(...  
)
```

Channel vs Flow: выбирать надо по семантике

“какая ментальная модель точнее”

Вопрос	Channel	Flow	StateFlow / SharedFlow
Что это?	Mailbox / очередь между coroutines	Декларативный поток значений	Разделяемый hot поток / состояние
Когда уместен?	Команды, задания, pipeline	Наблюдаемые данные и трансформации	UI state и fan-out событий
Что с повторной подпиской?	Последнее значение не обязано сохраниться	Cold Flow пересобирает upstream	Новый подписчик может получить current/replay
Ключевой риск	Переусложнить логику событий	Случайно собирать слишком часто	Неправильно выбрать replay/область жизни

Offtopic - state

State - как правильно перевести?

State

- state = набор актуальных значений
- state = одно из дискретных состояний автомата
- data class UiState - state as snapshot
- sealed class UiState - state as finite mode

State

```
data class PlayerUiState(  
    val title: String,  
    val progress: Int,  
    val isPlaying: Boolean,  
    val volume: Float,  
)
```

```
sealed interface DownloadState {  
    data object Idle : DownloadState  
    data object Downloading : DownloadState  
    data object Success : DownloadState  
    data class Error(val message: String) : DownloadState  
}
```

Главная мысль

есть state-данные и state-переходы

- Есть два типа состояния UI
- A. Product state / render state
 - Данные, нужные для отрисовки экрана.
 - Обычно это data class.
- B. Mode / phase
 - Крупный взаимоисключающий режим экрана.
 - Часто это sealed class.

Выбор data class если:

- состояние состоит из нескольких независимых кусков, которые могут сочетаться.
 - Например:
 - isRefreshing
 - items
 - query
 - selectedTab
 - errorMessage

sealed class использовать, если

- есть взаимоисключающие режимы.
- Например:
 - экран авторизации: Unauthenticated / Authorizing / Authenticated / Error
 - splash: Checking / NeedLogin / Ready
 - full-screen загрузка: Loading / Empty / Content / Error

End